DeepCQ: A Two-Stage Deep-Surrogate Framework for Lossy Compression Quality Prediction

Khondoker Mirazul Mumenin*, Robert Underwood†, Dong Dai‡, and Jinzhen Wang*
Sheng Di†, Zarija Lukić§, and Franck Cappello†
*University of North Carolina at Charlotte, Charlotte, NC, USA
{kmumenin,jwang96}@charlotte.edu
†Argonne National Laboratory, Lemont, IL, USA
{runderwood,sdi1}@anl.gov, cappello@mcs.anl.gov
‡University of Delaware, Newark, DE, USA
dai@udel.edu
§Lawrence Berkeley National Laboratory, Berkeley, CA, USA
zarija@lbl.gov

***Abstract*—Error-bounded lossy compression techniques have become essential for scientific data management and analytics as modern simulations and instruments generate ever-increasing data volumes. However, assessing post-compression data quality remains computationally expensive due to the cost of compression, decompression, and metric evaluation, hindering compressor selection and tuning in large-scale scientific workflows. In this paper, we present DeepCQ, a modular two-stage deep-surrogate framework that produces accurate compression quality prediction across widely used lossy compressors and quality metrics, enabling users to bypass costly compression operations during configuration selection. Evaluation results demonstrate that DeepCQ achieves an average prediction error below 10% in 158 out of 165 evaluated cases, significantly outperforming existing approaches. In addition, DeepCQ delivers high-throughput GPU inference, enabling fast online compression selection in parallel scientific applications. By allowing users to efficiently navigate large compressor configuration spaces based on user-preferred quality metrics, DeepCQ significantly reduces I/O overhead while preserving data fidelity in exascale scientific workflows.**

I. Introduction

With the advent of exascale High-Performance Computing (HPC) systems, scientific simulations generate unprecedented volumes of multi-dimensional data, posing significant challenges for storage, transmission, and analysis. For example, the Climate Earth System Model (CESM) [1] can produce petabytes of data, while large-scale Nyx simulations generate terabytes per snapshot [2]. Despite improved simulation fidelity, the sheer volume of data often leads to the loss of critical physics during subsequent analysis.

Error-bounded lossy compression techniques [3]–[12] are widely used to mitigate this challenge by reducing data size while enforcing a user-defined error bound. However, the effectiveness of compression depends critically on the choice of this bound. While it guarantees numerical accuracy, it does not directly correspond to application-relevant quality metrics such as Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index Measure (SSIM) [13], [14], which are often more meaningful to domain scientists.

This mismatch introduces a fundamental challenge: users control error bounds but must reason about quality metrics that are not directly predictable from them. The relationship between error bounds and quality metrics is nonlinear and varies across datasets and compressors [15], forcing users to rely on iterative trial-and-error processes involving repeated compression, decompression, and metric evaluation.

Fig. 1 illustrates the overhead of such an iterative process. Evaluating a single metric (e.g., SSIM) across a modest number of error bounds can take over 1200 seconds for a single compressor. This cost further scales with the number of compressor candidates, error bound configurations, and quality metrics of concern, making exhaustive exploration impractical, particularly in time-sensitive workflows.

The problem is further exacerbated by the growing diversity of lossy compressors. There is an increasing number of lossy compressors, or variants of a compressor family, developed with specialties tailored for specific data characteristics [16]–[18], causing the search space to enlarge. Therefore, there is a critical need for an efficient and scalable approach to predict compression quality across compressors and error bounds without exhaustive evaluation.

Motivated by this necessity, we propose DeepCQ, a two-stage deep surrogate framework for compression quality prediction. DeepCQ accurately predicts common quality metrics for widely used lossy compressors. The modular design supports modular integration of new compressors and quality metrics by adding prediction heads without retraining the shared backbone, which sets itself apart from existing solutions [19]–[21]. The technical contributions of DeepCQ are summarized as follows:

- We design a deep surrogate framework that **accurately** predicts compression quality metrics across multiple compressors, quality metrics and datasets, achieving less than 10% error in 158 out of 165 cases.
- We propose a two-stage architecture that decouples feature extraction from quality prediction, enabling **efficient** shared representations reuse across data fields that significantly

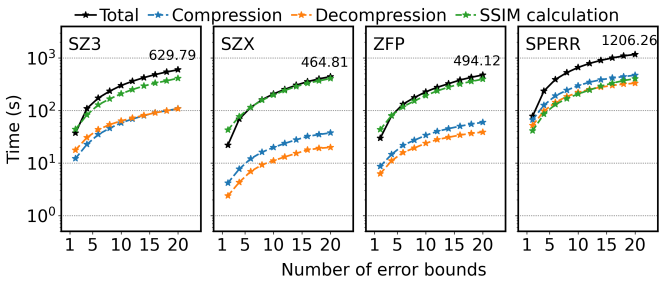


Fig. 1: Cumulative time to examine compressed data quality (SSIM) across 20 error bounds. Dataset: Dark matter density of Nyx; dimension: $512 \times 512 \times 512$.

reduces training overhead (1.81× for the Nyx application).

- We incorporate a Mixture-of-Experts (MoE) design into the prediction head, improving prediction **robustness** against regime-dependent compression quality variation, particularly in simulations where timestep evolution produces varying compression behavior.
- We demonstrate the **scalability** of `DeepCQ` in realistic parallel workflows, showing the advantages in both application throughput and compression efficiency compared to compression-based baselines.

To the best of our knowledge, `DeepCQ` is the first surrogate-based compression quality prediction framework that unifies prediction across multiple compressors and quality metrics through a shared backbone design, enabling extensibility without full model retraining. `DeepCQ` is designed as an application-specific predictor: the backbone is trained once per scientific application and shared across compressors, metrics, and data fields within that application, intentionally trading cross-application generality for accuracy and training efficiency. `DeepCQ` allows domain scientists to make informed decisions about data compression through surrogate-based prediction, improving the balance between storage cost and data fidelity in exascale scientific workflows.

The remainder of the paper is organized as follows. Section II reviews related work. Section III presents the design of `DeepCQ`, followed by evaluation in Section IV. Section V discusses limitations and future work, with conclusions in Section VI.

II. Related Work and Motivation

A. Error-bounded Lossy Compression

Error-bounded lossy compression techniques have been the predominant focus for data reduction, as they effectively reduce the volume of scientific data while respecting the users' accuracy requirements. Based on the way of data decorrelation, lossy compression can generally be divided into *prediction-based* and *transform-based*.

Prediction-based lossy compressors essentially achieve compression by removing data redundancy using predictive models. SZ2 [3], [6]–[8], an example of prediction-based compression techniques, starts by predicting each data point using a curve-fitting model based on the neighboring data point. Then, it performs quantization on the difference between the original and predicted value, followed by applying Huffman coding and lossless compression techniques to reduce the bit rate of each quantization code. Its successor, SZ3 [16], [22], [23], further enhances the compression performance by introducing a modular design with interchangeable components (predictors, quantization methods, encoders, and lossless backends) so that the compressor can be tailored toward input datasets and achieve the best-fit compression ratio. SZx [24] is another SZ family variant that primarily focuses on providing ultra-fast compression speed. It is designed to use lightweight operations and is optimized for CPU and GPU architectures while preserving relatively high compression ratios.

On the other hand, transform-based lossy compressors usually adopt data transform schemes to remove data redundancy. Then, various encoding methods are used to represent the transform coefficients. For example, ZFP [4] first adopts a non-orthogonal transform on the data within each block, then a customized variable-length embedded encoding to represent transform coefficients one bit-plane at a time. Due to the block-wise design, ZFP offers superior scalable performance and random access to compressed data blocks. SPERR [25] similarly adopts a biorthogonal wavelet transform on chunks of input data. Then it adopts a SPECK [26], [27]-an inspired scheme to encode the outliers efficiently. SPERR also offers scalable execution capability as well as point-wise error tolerance, which is often desired by many scientific applications.

B. Compression Ratio and Quality Prediction

In practice, it is critical for users to understand how a lossy compressor with a given error bound impacts the final compression ratio and the quality of the resulting compressed data. As described, this is a non-trivial task. Many efforts have been made to address this challenge. We discuss existing approaches and their limitations, motivating our work.

■ ***Compression Ratio Prediction:*** First, users need to understand how much space can be saved with certain lossy compressors and error bounds. Existing attempts mainly achieve this in two steps: *1) extracting meaningful features from input data that correlate well with compression ratio* and *2) making accurate predictions based on the extracted features*.

The attempts on data feature extraction can be further categorized into two strategies: *1) autonomous extracting from down-sampled data* and *2) assembling statistical data features by hand*. Lu [28], Wang [29], [30], Tao [19] and Khan [21] fall into down-sampling-based category. They also investigated the impact of different sampling methods and concluded that the prediction could be more accurate if their sampling method matches the way compressors process input data [21], [28]. For example, as ZFP performs local transform and bitplane encoding in 4^d blocks, sampling at the same size can yield unbiased compression ratio prediction.

On the other hand, Qin [31], Underwood [32], Krasowska [33] and Ganguli [34] hand-crafted the data features. The early work by Qin [31] relied on simple data features (mean, variance, standard deviation, quartiles, histograms) and compressor-specific metrics (curve-fitting hit ratio, Huffman

tree size, etc.) to help predict the compression ratio. More recent work like Underwood [32] designed truncated SVD and quantized entropy as predictors for compression ratio and achieved high prediction accuracy. Krasowska [33] further showcased on 2D datasets that data correlation could be a good feature in compression ratio prediction. Ganguli [34] further proposed hand-picked data features (Spatial Diversity, Spatial Correlation, Generic Distortion Measurement, Coding Gain, and Spatial Smoothness) that have been studied to have a strong correlation with data compressibility [32].

After extracting the data features, the next step is to make accurate predictions. Lu [28], Tao [19], Wang [29], [30] and Khan [21] explored statistical models, based on the empirical study on the correlation between compression ratio and multiple compression metrics (prediction hit ratio, Huffman tree size, distribution of SZ prediction error). Meanwhile, the work by Qin [31], Underwood et al. [32] and Ganguli [34] adopted data-driven approaches like regression models or Deep Neural Network (DNN) models for compression ratio prediction.

■ ***Compression Quality Prediction:*** Recently, as the quality of the compressed data has become increasingly critical for domain scientists, a few new studies have emerged in predicting data quality-relevant metrics, such as SSIM or PSNR, in addition to compression ratio. The strategies adopted, however, are not fundamentally different. For example, Jin's work [20] models compression ratio, PSNR, and SSIM via a mathematical model specially designed for prediction-based compressors. Such a model also leverages the down-sampled data. A recent work by Mumenin et al. [35] adopts a DNN-based approach that takes raw scientific data and error-bound values as input to predict compression quality metrics such as PSNR, SSIM, and compression error autocorrelation (AC). It is among the first to learn data representations directly without relying on externally engineered features. However, the model requires training for each specific dataset, compressor and quality metric, which poses challenges for efficient model training and deployment for compression configuration tuning.

C. Challenges and Motivation

Among existing approaches that are mentioned above, many focus on a specific prediction task rather than being a general-purpose solution. For example, Khan's work [21] focuses solely on compression ratio prediction; Tao's work [19] supports only a rough compression ratio prediction to enable selection between SZ2 and ZFP; Jin's work [20] only supports SZ3. While Jin's work claims support for PSNR and SSIM prediction, its implementation [36] primarily targets compression ratio. As a result, these methods often work in silo, with focuses on specific tasks. Users would need to carefully examine what predictions can be done for their application and how accurate the results are. The fundamental limitations of these approaches come from the hand-crafted prediction models. They either rely on compressor-specific implementations or quality-metric-specific ones, which does not work on other cases without a redesign.

Learning-based approaches like QualityNet [35], on the other hand, adopts a black-box DNN surrogate which can potentially be trained towards individual prediction tasks. However, the monolithic retraining incurs excessive overhead and management costs.

Apparently, none of the existing compression quality prediction methods provide a unified solution for commonly used lossy compressors and quality metrics. Motivated by such a gap, ***we aim to develop a surrogate-based compression quality prediction framework, with the ability to work for most commonly used lossy compressors and quality metrics, while being as efficient as possible.*** The key idea originates from the observation that the data features shared by the prediction tasks for the same data field would be highly redundant and reusable. Therefore we can design the overall framework with the aim of learning shared data features and reuse for downstream prediction tasks.

III. Compression Quality Prediction Framework

We first formulate the compression quality prediction problem and then present the design of our framework. Key terms are summarized in Table I.

A. Problem Formulation

Compression quality metrics quantify the distortion introduced by lossy compression by referencing the original dataset [37]. Typically, these metrics are scalar values derived from differences between the original and decompressed data, providing a summary of compression errors for multi-dimensional scientific data.

Obtaining these metrics is computationally expensive, as it often requires compression, decompression, and metric evaluation on the full-scale data. On the other hand, these quality metrics are also highly dependent on data characteristics and compressors, making it hard to manually derive. Existing work [35] has tried black-box DNN models as surrogate to learn quality metrics from input data and compressor error bounds. However, due to significant inefficiency, such an approach is not practical across diverse quality metrics, lossy compressors, and datasets.

Our objective is to enhance the efficiency of deep-surrogate-based compression quality prediction across diverse quality metrics, lossy compressors, and scientific applications. In particular, we focus on three representative quality metrics: PSNR, SSIM, and Compression Ratio (CR)[1], where PSNR and SSIM quantify data distortion during compression, and CR measures reduction efficiency.

B. `DeepCQ` : A Two-Stage Approach

The central design principle of `DeepCQ` is to decouple data feature extraction from compression quality prediction, motivated by the known interplay among lossy compression,

[1]We extend the definition of compression quality metrics to include compression ratio, as it is an essential measurement for assessing data reduction performance.

TABLE I: Key terminologies used in this paper

Term	Description
PSNR	Peak Signal-to-Noise Ratio
SSIM	Structural Similarity Index Metric
CR	Compression Ratio
MAPE	Mean Absolute Percentage Error
DFE-NN	Data Feature Extraction Network
EFE-NN	Error bound Feature Extraction Network
Pred-NN	Prediction Network
MoE	Mixture of Experts

input data, and quality metrics. Formally, compression quality metrics can be expressed as:

$$\mathrm{QM} = f(\mathrm{X}, \mathrm{EB}, \mathrm{CP}),$$

where X denotes the input data, EB the error bound, and CP the compressor. While this mapping is highly nonlinear, empirical evidence from prior work [35] and our own experiments suggest that the influence of X can be factored through a lower-dimensional representation capturing data properties such as spatial correlation, smoothness, entropy, and local variability, all of which are independent of the downstream compressor.

Scientific simulation outputs often exhibit strong structural regularities governed by underlying physical processes, such as smooth regions, sharp gradients, and turbulent zones. These characteristics remain stable across compressors and error bounds within the same application, and thus admit a shared representation. In contrast, the mapping from these representations to observable compression outcomes is compressor- and metric-specific, reflecting algorithmic choices such as prediction schemes, transforms, quantization, and entropy coding.

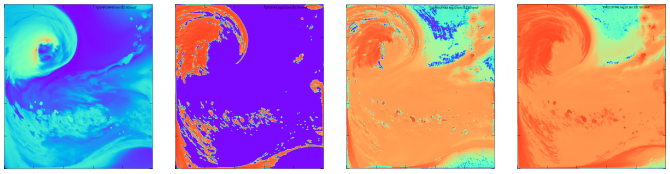
This observation motivates decomposing the quality prediction problem into two stages:

$$QM = g\big(\phi(X), \psi(EB)\big) \tag{1}$$

where $\phi(X)$ extracts compressor-agnostic data features, $\psi(EB)$ embeds the scalar error bound, and $g(\cdot)$ models the compressor- and metric-specific response.

This decomposition has three important implications. First, it enables *shared feature extraction across data fields*, since many fields share similar spatial structures within the same application (as illustrated in Fig. 2), making per-field retraining unnecessary. Second, it supports *modular extensibility*, where new compressors or quality metrics can be incorporated by training lightweight prediction heads without retraining the entire model. Third, it provides a natural foundation for handling *heterogeneous compression behavior* through customized prediction heads, allowing independent refinement of the feature extractor or prediction head without full-model retraining costs.

We demonstrate the two-stage framework in Fig. 3. During training, a task-agnostic feature extraction backbone is trained once per application and reused across all data fields, followed by lightweight metric prediction heads trained individually for specific tasks. In contrast to prior approaches that directly learn a monolithic mapping from (X, EB) to quality metrics,



(a) QVAPOR **(b)** QICE **(c)** QGRAUP **(d)** PRECIP

Fig. 2: Structural similarity across four fields of the Hurricane Isabel simulation.

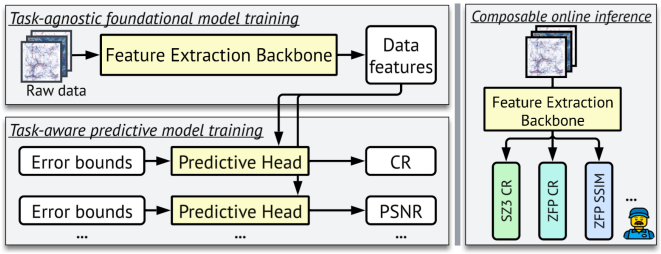


Fig. 3: Left: two-stage compression quality prediction framework during training. Highlighted parts represent the DNN models. Right: composable framework during inference.

`DeepCQ` exploits this decomposability to reduce training cost, improve robustness, and enable scalable deployment.

`DeepCQ` is designed as an *extensible, multi-compressor compression quality prediction framework*, rather than a universal model applicable to all data types and compression algorithms. In this work, we focus on *error-bounded lossy compressors operating on structured, multi-dimensional floating-point scientific data*, which represent a dominant class of production HPC workloads. Within this scope, `DeepCQ` naturally supports all compressor families (prediction-based and transform-based) and full-reference quality metrics.

`DeepCQ` is application-specific by design. The feature extraction backbone is trained per scientific application to capture domain and physics-dependent spatial characteristics shared across data fields and timesteps. This design choice intentionally trades global generality for improved accuracy, reduced training cost, and practical deployment in real workflows, where data distributions are tightly coupled to simulation physics. Therefore, `DeepCQ` does not aim to provide zero-shot generalization across unrelated applications.

The remainder of this section describes the feature extraction backbone, the metric prediction head, and the strategy for handling variation in compression quality, highlighting how their integration enables accurate, efficient, and robust compression quality prediction.

C. Feature Extraction Backbone

The feature extraction backbone, implemented as a Data Feature Extraction Network (DFE-NN), is designed to capture representative features from high-dimensional scientific data, forming the foundation for downstream compression quality prediction. Prior work by Mumenin et al. [35] demonstrated that CNN-based architectures are effective for extracting spatial features from 2D scientific datasets. Building on this insight, we explore whether more advanced model architectures can further enhance feature extraction for 3D data.

CNN-based models, such as ResNet [38]–[40] and U-Net [41], remain the dominant choice for 3D volumetric scientific data due to their efficiency in learning local spatial patterns and robustness under limited training data. Their convolutional inductive biases facilitate the capture of hierarchical spatial dependencies, enabling stable and effective feature extraction for metrics such as CR, PSNR, and SSIM even with relatively small datasets. In contrast, transformer-based architectures such as ViT [42] and ViViT [43] excel at modeling long-range dependencies through self-attention mechanisms, making them appealing for complex spatiotemporal data in domains like climate, astrophysics, and MRI [44]. However, these models typically require large-scale datasets to learn effective representations, which limits their practicality in data-scarce scientific workflows.

Since our backbone is trained per scientific application rather than as a large-scale foundation model, assembling datasets of sufficient scale is infeasible, making transformer-based architectures less suitable. Instead, we adopt a CNN-based backbone, which provides stronger inductive biases for learning spatial structure and achieves better sample efficiency in limited-data regimes. The backbone is trained once per application domain and then frozen for all downstream regression tasks. Unlike end-to-end approaches such as [35], where feature learning and prediction are jointly optimized, our setting demands representations that are inherently robust and transferable across multiple compressors and quality metrics. Compact architectures [45], [46] consistently fall short in this regard, as their limited capacity produces weak feature embeddings that are insufficient for accurate downstream regression.

To address this, we adopt a high-capacity CNN backbone based on ResNet-152, extended to 3D by replacing all 2D convolutional and pooling layers with their 3D counterparts. This design enables the model to capture spatial correlations across width, height, and depth, producing rich volumetric representations that support accurate and stable compression quality prediction.

D. Metric Prediction Head

As compression quality metrics are jointly influenced by the input data and the specified error bound, the metric prediction head needs to integrate features from both sources. While the input data features are extracted by the DFE-NN, an important design question arises: *how can a scalar error bound be effectively represented and fused with data features to enable accurate quality metric prediction?* To address this, we design the metric prediction head with two key components—the error bound Feature Extraction Network (EFE-NN) and the Prediction Network (Pred-NN).

■ ***Error bound feature extraction:*** In error bounded lossy compression workflows, the error bound is a single scalar value, offering limited representational capacity for capturing its complex influence on data quality. To overcome this limitation, we employ a learnable embedding strategy [47] by introducing the EFE-NN, which transforms the scalar error bound into a high-dimensional embedding. The EFE-NN consists of two multi-layer perceptrons (MLPs) with the dimension of 128 and 256, respectively. This lightweight yet expressive architecture effectively encodes the scalar error bound into semantically rich representations that enhance downstream metric prediction accuracy.

■ ***Prediction feature combination:*** Once the data and error bound features are extracted, they are fused for metric prediction. Rather than complex fusion methods that risk distorting learned representations [48], we adopt a lightweight Siamese-inspired fusion strategy [49], where both features are jointly fed into Pred-NN while preserving feature integrity.

E. Taming variation in compression quality

The metric prediction head provides a simple and effective mapping from combined data and error bound features to compression quality. However, in practice, prediction accuracy degrades under heterogeneous data regimes. This effect is particularly pronounced in the Nyx cosmology simulation.

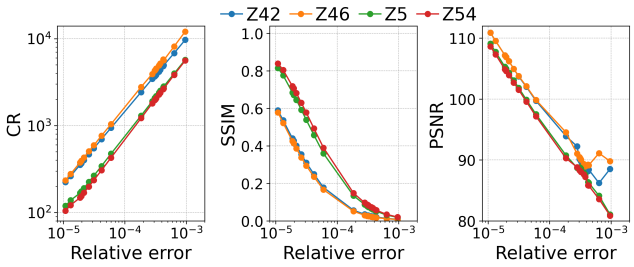
In Nyx, timestep evolution is controlled by the redshift parameter z.Snapshots are recorded at discrete checkpoints such as $z42$,$z46$, $z5$ and $z54$, corresponding to redshift values 4.2, 4.6, 5.0, and 5.4. As the simulation evolves, redshift decreases while intermediate integration steps introduce substantial structural changes in the data. These changes lead to distinct compression behaviors across timesteps. As shown in Fig. 4, CR, PSNR, and SSIM form two clearly separated groups, namely $\{z42, z46\}$ and $\{z5, z54\}$, indicating regime-dependent compression characteristics. This heterogeneity poses a fundamental challenge, as a single prediction head must approximate a mapping from $(\phi(X), \psi(EB))$ to quality metrics across regimes with distinct input–output relationships. Thus, it learns an averaged function over incompatible mappings, introducing systematic bias near regime boundaries. The effect is most visible in CR prediction, where small changes in data characteristics lead to different compression responses. This limitation is structural rather than a matter of model capacity. Regardless of the depth or width of a single MLP, the model remains constrained to a single shared mapping over a multi-regime input space. Consequently, the model produces a compromised approximation across regimes, resulting in over-smoothing and systematic bias.

To address this, we introduce a Mixture-of-Experts (MoE) architecture [50] on top of the Pred-NN, as shown in Fig. 5. MoE is suitable when the mapping from features to targets is regime-dependent rather than globally smooth [50]. Instead of enforcing a single function, MoE decomposes the problem into multiple experts. Each expert models a subset of the input space, while a routing network combines their outputs.

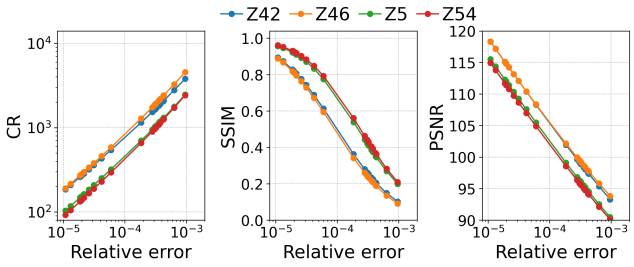
The MoE architecture replaces $g(\cdot)$ in Eq. 1 with a mixture of K specialized experts, giving

$$\mathbf{QM} = \sum_{i=1}^{K} \alpha_i(\phi(\mathbf{X}),\, \psi(EB))\, \beta_i(\phi(\mathbf{X}),\, \psi(EB)) \quad (2)$$

where $\beta_i(\cdot)$ denotes the i-th expert network and $\alpha_i(\cdot)$ is the corresponding gating weight satisfying $\sum_{i=1}^{K} \alpha_i = 1$. Our

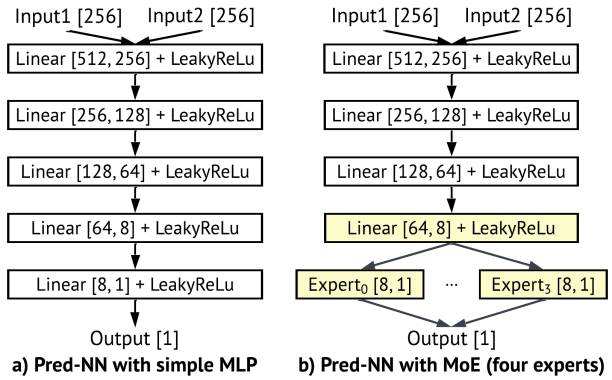


(a) SZ3



(b) SPERR

Fig. 4: Compression quality of Baryon density across timesteps.



a) Pred-NN with simple MLP **b) Pred-NN with MoE (four experts)**

Fig. 5: The architecture of Pred-NN. Highlighted components in b) represent the MoE design.

design uses four parallel linear experts ($K = 4$) and a soft gating network. The gating network operates on the shared representation and applies layer normalization and dropout. It produces expert logits, which are converted into mixture weights using softmax. Each expert produces an independent prediction, and the final output is computed as a weighted sum.

We adopt soft gating instead of top-K selection. Sparse routing can lead to expert collapse in low-data settings, where only a few experts receive gradients [51], [52]. Soft gating assigns non-zero weights to all experts, which stabilizes training and promotes balanced specialization. To further reduce early bias, experts are initialized with small parameter differences, and the gating network is initialized conservatively.

While MoE improves robustness under regime-dependent variation, it is not universally advantageous. When the mapping is smooth and well-behaved, as commonly observed for PSNR, a single MLP achieves comparable performance. The empirical benefit of MoE is validated in Section IV-F.

IV. Evaluations

In this section, we evaluate the performance of DeepCQ. We first assess prediction accuracy across individual data fields (Section IV-B). Next, we evaluate the overhead and scalability of DeepCQ in parallel computing environments and compare it against a compression-based baseline approach (Section IV-C). We then compare DeepCQ's prediction accuracy with existing compression quality prediction methods (Section IV-D). Finally, we present ablation studies to validate the design choices of our model (Section IV-F).

A. Experimental Setup

■ ***Testbed:*** All experiments, including compression, quality metric computation, and DNN model training, are conducted on a local cluster equipped with 48 Intel Xeon Silver 4214 CPUs and 8 NVIDIA Quadro RTX 5000 GPUs.

■ ***Software:*** We evaluate five widely adopted lossy compressors (SZ2 [3], [6]–[8], SZ3 [16], [22], [23], SZx [24], ZFP [4], and SPERR [25]), which collectively cover diverse compression philosophy and offer varying trade-offs between compression ratio and reconstruction quality. All compression tasks are executed via the LibPressio framework [36], which provides a unified interface and standardized performance metrics. Quality metrics are computed using the Quick Compression Analysis Toolkit (QCAT) [53] to ensure consistent and reproducible evaluation results.

■ ***Datasets:*** We evaluate DeepCQ on real-world scientific data from various domains, offered by the Scientific Data Reduction Benchmark [54], including Nyx cosmology simulation [55], Hurricane Isabel climate simulation [56], and Miranda turbulence simulations [57]. The datasets description is included in Table II. For the Miranda, the full-scale dataset is partitioned into subdomains of size $256 \times 384 \times 384$ to create disjoint training and test samples.

■ ***Compression configurations:*** Compression quality is highly sensitive to the configurations, particularly the choice of error bounds. In this study, we configure error bounds to balance between achieving meaningful compression ratios and preserving sufficient fidelity for downstream scientific analysis. Guided by domain experts, we set the relative error bound range for each application as follows: Nyx: 10^{-5} to 10^{-3}, Hurricane: 10^{-5} to 10^{-2}, Miranda: 10^{-4} to 10^{-2}.

■ ***Model training and testing configurations:*** We employ a two-stage training procedure aligned with the modular design of DeepCQ. In the first stage, a shared feature extraction backbone is pretrained for each application to learn compact spatial embeddings from the input scientific data. This stage is task-agnostic and does not use compressor outputs, error-bound labels, or compression-quality targets; therefore, all available raw data within an application is used for backbone pretraining. In the second stage, the pretrained backbone is frozen, and task-specific prediction heads are trained independently for each compressor–metric prediction task.

Phase 1: Backbone Training Training the feature extraction backbone on scientific data presents three key challenges: (i) acquiring sufficiently diverse timesteps is often infeasible [58];

TABLE II: Datasets used in this work.

Application	# Timesteps	# Fields	Dimension	Size
Nyx	4	4	$512 \times 512 \times 512$	8 GB
Hurricane	48	6	$100 \times 500 \times 500$	26.8 GB
Miranda	1	1	$3072 \times 3072 \times 3072$	216 GB

(ii) the large scale of scientific datasets complicates efficient mini-batch data loading [59]; and (iii) varying simulation output dimensions (e.g., Nyx data ranging from 512^3 to 2048^3 [60]) hinder direct ingestion into CNN-based models. Unlike standard vision pipelines, large-scale pretraining datasets for scientific data are not readily available due to the lack of sufficiently diverse and standardized benchmarks. As a result, the backbone must be trained directly on application-specific data.

To handle varying spatial resolutions and reduce memory overhead, we adopt a random block-sampling strategy that extracts fixed-size 64^3 subvolumes from the full-resolution data, treating each block as an independent training instance. The use of a fixed block size provides a unified input representation across datasets with different spatial dimensions and enables a consistent training pipeline. The block size is selected by balancing three considerations: the number and diversity of training samples that can be generated from the available simulation outputs, the spatial context required to capture relevant data features, and the GPU memory required for mini-batch training. Empirically, smaller blocks, such as 16^3 and 32^3, provide insufficient spatial context for the downstream regression task and degrade the learned local representations. Conversely, increasing the block size beyond 64^3 yields only marginal improvements in prediction accuracy while substantially increasing memory consumption and computational overhead. Therefore, 64^3 offers a practical balance among sample diversity, spatial context, and computational efficiency. Using this strategy, we generated 3,000 blocks from three Nyx timesteps, 5,000 blocks from 25 Hurricane timesteps, and 6,000 blocks from the Miranda dataset to train the backbone efficiently.

To learn transferable feature representations from these blocks, we adopt an autoencoder-based pretraining strategy. Autoencoders are well suited for this purpose, as they learn compact and generalizable latent representations by reconstructing the input without requiring labeled supervision [61], [62]. The encoder learns to capture dominant spatial structures of the input volumetric data, while the decoder enforces representational completeness through input reconstruction. Once pretraining is complete, the decoder is discarded and the encoder is retained as the feature extraction backbone. Its weights are then frozen for all downstream metric prediction tasks, ensuring that the learned representations remain unbiased toward any specific compressor or quality metric and are transferable across all prediction heads without retraining [63].

The backbone is pretrained for 250 epochs using the Adam optimizer with an initial learning rate of 0.01, decayed via cosine annealing scheduling, and optimized under an MSE reconstruction loss.

Phase 2: Prediction Head Training Error bounds span several orders of magnitude (e.g., 10^{-5} to 10^{-1}), which can impede model convergence as model gives disproportionate attention to the large errors, effectively ignoring the much smaller ones. To mitigate this issue, we apply a logarithmic normalization to error bounds before feeding into EFE-NN. Each prediction head is trained for 150 epochs using the Adam optimizer, a learning rate scheduler initialized at 0.01, and RMSE loss. To construct the training data for the metric-prediction heads, we sample error bounds for each available field and timestep. We use 500 samples for Nyx, 100 for Hurricane, and 20 for Miranda. The number of samples differs across applications because of variations in field and timestep availability.

To ensure fair evaluation, all datasets are divided into disjoint training and testing subsets. For the Nyx dataset, the first three timesteps ($z42$, $z46$, $z5$) are used for training and the final timestep ($z54$) for testing, enforcing a strict temporal holdout. For the Miranda dataset, non-overlapping data chunks are partitioned by index parity, with even-indexed samples used for training and odd-indexed samples for testing, ensuring disjoint splits. For the Hurricane dataset, timesteps 1–25 are used for training, and all subsequent timesteps are reserved for testing, with timestep 26 used for evaluation.

Across all datasets, these protocols enforce non-overlapping train–test separation, preserve intrinsic temporal or spatial structure, and evaluate performance under realistic distribution shifts while maximizing the effective use of limited data.

■ ***Prediction accuracy assessment:*** We use Mean Absolute Percentage Error (MAPE) to measure the average percentage error across all error bounds, data samples, and timesteps:

$$\text{MAPE} = \frac{1}{n} \sum \left| \frac{\text{Orig} - \text{Pred}}{\text{Orig}} \right| \times 100$$

B. Prediction Accuracy on Individual Fields

Table III summarizes the prediction performance of `DeepCQ` across 5 compressors, 3 quality metrics, and 11 data fields. `DeepCQ` achieves MAPE below 10% in 158 out of 165 cases, demonstrating robust predictive accuracy across diverse compressor–metric–field combinations. The model performs particularly well for PSNR and SSIM, while CR exhibits higher variability and remains comparatively more challenging to predict. The modular design, where each metric is modeled independently, reduces cross-objective interference and stabilizes prediction under heterogeneous tasks. Fig. 6 highlights representative failure cases with MAPE exceeding 10%, revealing two distinct sources of error.

The first is locally irregular compressor response. On QICE data, the CR curve of SZ3 exhibits localized fluctuations near a relative error bound of 3×10^{-3}. The surrogate captures the overall trend but cannot reproduce this abrupt local deviation, as the model learns a continuous mapping whereas the true compressor response contains non-smooth transitions. Denser sampling near such transition regions could reduce this error at modest additional training cost.

The second is temporal distribution shift. On QRAIN data, the predicted SSIM curves for SZ3 and SPERR capture

TABLE III: MAPE of compression quality prediction using DeepCQ (in %).

Application	Fields	SZ2			SZ3			SZx			ZFP			SPERR		
		CR	PSNR	SSIM	CR	PSNR	SSIM	CR	PSNR	SSIM	CR	PSNR	SSIM	CR	PSNR	SSIM
Nyx	Baryon density	5.33	0.74	6.21	5.82	1.17	9.05	9.64	1.74	6.40	9.45	1.08	6.52	4.25	3.02	5.92
	Dark matter density	4.96	1.44	4.45	3.54	1.18	2.84	3.10	1.70	3.29	5.07	1.60	1.86	3.69	2.33	6.23
	Temperature	3.54	3.61	0.94	3.13	3.71	1.10	2.34	2.84	1.77	4.51	3.30	1.63	1.51	3.72	0.67
	Velocity x	1.38	1.59	2.13	2.20	1.66	2.18	1.89	1.97	2.10	2.84	1.57	2.00	4.04	2.32	2.04
Hurricane	CLOUD	3.26	2.25	2.62	9.37	2.00	1.82	9.63	1.63	1.69	5.65	2.03	2.23	3.03	0.92	2.07
	QVAPOR	9.25	2.54	4.08	9.86	3.53	2.60	2.78	2.78	4.23	7.40	2.04	4.29	3.46	3.59	5.51
	PRECIP	4.95	2.12	4.12	12.87	1.78	2.62	4.33	2.70	1.54	6.00	2.09	4.59	4.72	1.85	5.40
	QGRAUP	5.18	1.56	4.75	8.15	1.08	2.90	3.71	1.08	6.40	6.70	1.27	7.59	5.24	1.35	6.03
	QICE	12.41	2.53	2.25	12.59	2.38	2.12	6.64	2.47	2.56	7.92	3.31	2.91	6.62	2.62	2.27
	QRAIN	5.63	0.57	7.51	8.15	1.58	12.69	1.56	1.48	1.30	5.27	1.27	8.74	8.05	1.33	12.43
Miranda	Density	11.05	1.54	0.89	9.81	0.64	0.19	3.52	1.66	0.39	12.75	1.88	0.50	4.09	0.55	0.17

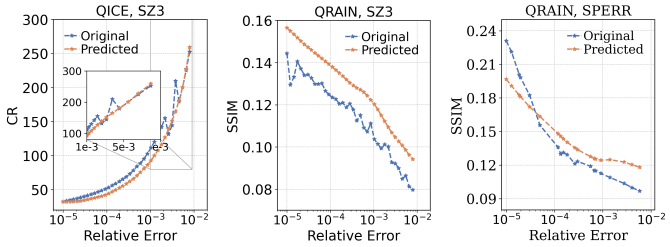


Fig. 6: Three cases of less accurate prediction, each representing a kind of challenge for compression quality prediction.

the overall SSIM degradation trend but exhibit a systematic offset from true values. Fig. 7 shows that SSIM distributions shift noticeably over time: earlier training timesteps span a wider range including higher-SSIM regimes, while later test timesteps concentrate in a lower and narrower range. This confirms that the relationship between error bounds and quality metrics in the QRAIN field is non-stationary across timesteps, shifting between the training and test regimes.

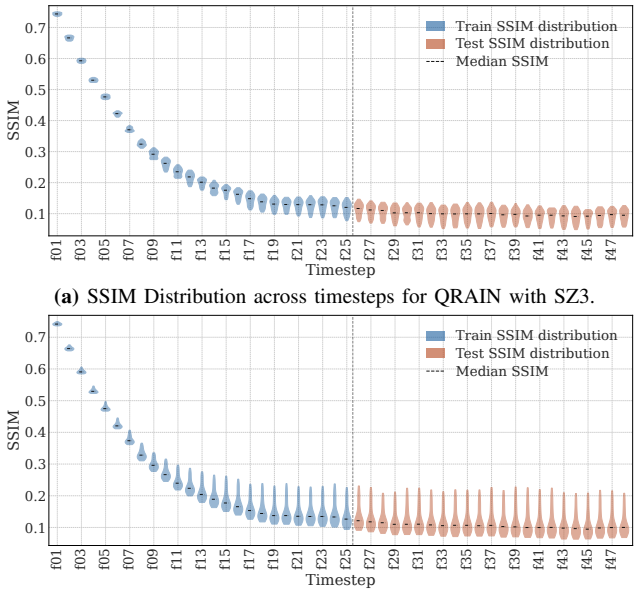
These observations in prediction bias highlight the need for more customized solutions. Specialized prediction heads can better capture compressor-specific behaviors that a generalized Pred-NN may fail to model accurately. For applications exhibiting significant temporal data drift, continuous validation is needed during long-term operation.

C. Use Case: Scalable Online Compressor Selection

We evaluate DeepCQ in a controlled parallel setting that reflects scientific simulation workflows, where data is generated across multiple MPI ranks and compression decisions must be made efficiently. We focus on isolating the overhead of compression configuration selection rather than end-to-end I/O performance.

In conventional workflows, selecting a compressor and its configuration requires exhaustively evaluating candidates through repeated compression, decompression, and quality assessment, incurring significant per-rank overhead. DeepCQ replaces this process with a prediction-driven approach, performing a single feature extraction followed by lightweight inference to evaluate all candidates.

We use the Hurricane application under weak scaling, where each rank processes one simulation output (95.3 MB,



(a) SSIM Distribution across timesteps for QRAIN with SZ3.

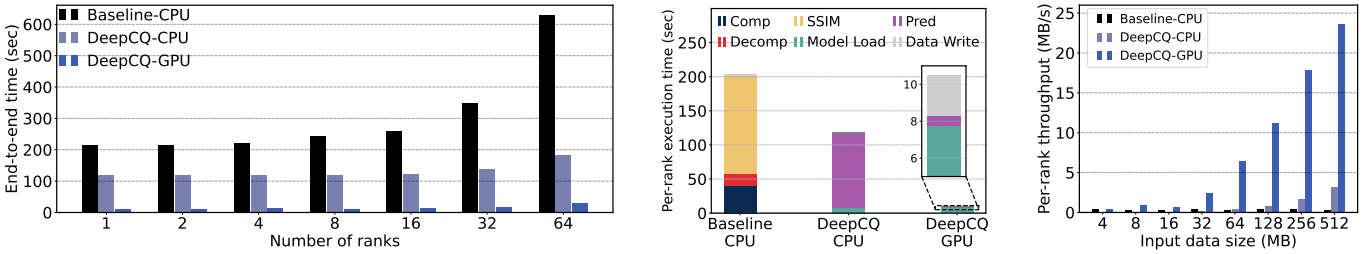
(b) SSIM Distribution across timesteps for QRAIN with SPERR.

Fig. 7: Temporal distribution shift in SSIM values for the QRAIN field under SZ3 and SPERR compression.

500×500×100), for a total of 5.95 GB per iteration. The objective is to select the configuration that maximizes CR×SSIM across 4 compressors (SZ3, SZx, ZFP, SPERR) and 10 relative error bounds. The baseline performs exhaustive evaluation via full compression and quality computation, whereas DeepCQ evaluates all candidates through a single forward pass, substantially reducing configuration selection overhead.

■ ***Parallel performance:*** Fig. 8(a) reports the execution time of DeepCQ (CPU and GPU) and the baseline as the number of ranks increases from 1 to where each rank receives a distinct 500×500×100 chunk. The baseline exhibits limited scalability due to the repeated cost of compression and quality evaluation across all candidate configurations.

In contrast, DeepCQ replaces compression-based evaluation with lightweight inference, significantly reducing configuration selection overhead. CPU-based DeepCQ consistently outperforms the baseline, while GPU-based DeepCQ further reduces decision latency through batched inference. At 64 ranks, DeepCQ processes 5.95 GB of data in under 32 seconds, corresponding to an effective prediction throughput of approximately 200 MB/s. This throughput reflects only the cost



(a) End-to-end execution time for compression selection. **(b)** Single-rank execution time. **(c)** Single-rank scalability w. data size.

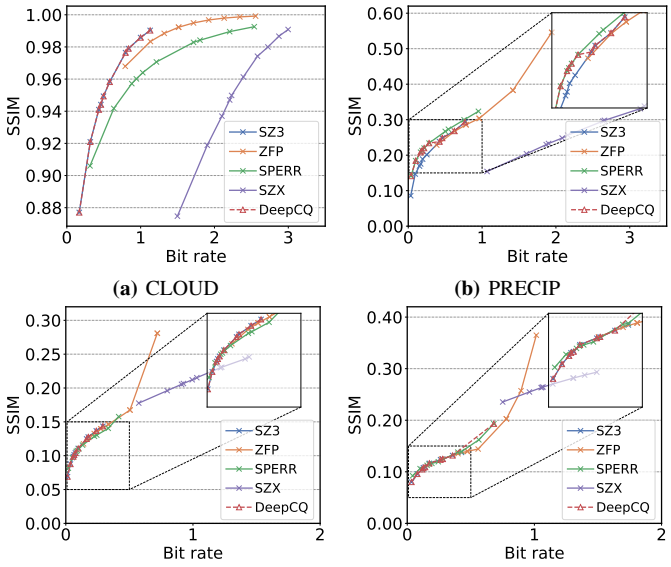
Fig. 8: Execution time and throughput of DeepCQ vs. Baseline. Hardware: 48 Intel Xeon Silver 4214 CPUs, 8 Nvidia Quadro RTX 5000 GPU; Dataset: CLOUD (95.3 MB/rank in left and middle), Baryon Density (right).

of compression configuration selection and is independent of the underlying compression or I/O bandwidth. This efficiency is enabled by computing feature representations once per data field and reusing them across all candidate configurations, avoiding redundant computation during configuration selection. As a result, DeepCQ enables efficient evaluation of multiple compression options with minimal additional cost.

■ ***Prediction overhead:*** Fig. 8(b) decomposes execution time on a single rank. For the baseline, the dominant cost arises from repeated compression and SSIM computation across all candidate configurations. DeepCQ eliminates this overhead by avoiding explicit compression during the selection phase. CPU-based inference reduces overall latency but remains constrained by single-core performance. In contrast, GPU-based inference introduces minimal additional latency relative to the compression decision process. We note that GPU acceleration is optional, and DeepCQ can operate in CPU-only environments when necessary.

■ ***Scaling with input data size:*** Fig. 8(c) shows the scaling throughput with respect to input data size on a single rank. In this particular case, we partition Baryon Density data of Nyx application into smaller data chunks with the sizes ranging from 4 MB to 512 MB. Clearly, DeepCQ-GPU scales the best and achieves the best performance across all data sizes, once again showcasing the capability of DeepCQ on delivering fast online compression selection. On the CPU side, baseline with exhaustive compression is more beneficial when data size is less than 64 MB, as DeepCQ-CPU spends more time on DNN inference. We would like to point out that, when data size is smaller than 4 MB, baseline will surpass the DeepCQ-GPU due to the model loading overhead. However, such cases is not meaningful in any real-world workflows.

■ ***Compression efficiency:*** Fig. 9 demonstrates the compression efficiency achieved by DeepCQ as compared to using default compression policies. We compare the achieved rate-distortion of using the best-fit compressor identified by DeepCQ, with SZ3, ZFP, SPERR and SZx, across 4 data fields of Hurricane application, over 10 error bounds. It is clear that DeepCQ is able to identify near-optimal compression configurations under each case, so that it always maintains the highest SSIM under the same bit rate. In particular for CLOUD dataset, where SZ3 consistently outperforms the rest compressors, DeepCQ also consistently selects SZ3 as



(a) CLOUD **(b)** PRECIP **(c)** QGRAUP **(d)** QRAIN

Fig. 9: Rate-distortion of DeepCQ vs compressors. Contributed to the accurate prediction of CR and SSIM, DeepCQ identifies the near-optimal compressor to maximize the rate-distortion.

the best-fit compressor. Among 4 data fields, 4 compressors and 10 error bounds, DeepCQ correctly identifies the best-fit compressor in 153 out of 160 cases. Under limited cases where DeepCQ does not find the optimal solution, the achieved rate-distortion is very close to the optimum, demonstrating the robust capability of online compressor selection based on users' requirements.

TABLE IV: MAPE comparison between DeepCQ and existing CR prediction approaches (in %)

Apps	SZ2		SZ3				ZFP		
	Tao19	DeepCQ	Khan23	Tao19	Jin22	DeepCQ	Khan23	Tao19	DeepCQ
Nyx	98.03	4.04	43.20	84.72	38.07	4.02	0.03	3.94	5.42
Hurricane	73.02	6.92	73.57	92.69	38.82	9.83	77.44	89.23	6.49
Miranda	86.73	11.05	84.98	95.61	51.96	9.81	77.96	79.32	12.75

D. Comparison with Existing Methods

Table IV reports the MAPE of DeepCQ compared with existing compression quality prediction methods, including Tao19 [19], Khan23 [21], and Jin22 [20] for SZ2, SZ3, and ZFP. For each application, we report the average MAPE across

all data fields. Results show that DeepCQ achieves the lowest prediction error in 26 out of 27 cases, with the only exception on ZFP CR prediction, where Khan23 and Tao19 outperform DeepCQ on the Nyx datasets. This is attributed to ZFP's block-wise compression mechanism, which produces highly regular patterns. In this case, sampling-based approaches can capture the CR well when the sampling is performed in align with ZFP's block-wise scheme.

It is important to note that Tao19 is primarily designed for online compressor selection between SZ2 and ZFP. As such, it focuses on providing coarse, order-of-magnitude estimates of CR, rather than precise prediction. Furthermore, due to algorithmic differences between SZ2 and SZ3, its performance does not directly transfer, leading to reduced accuracy when applied to SZ3.

Khan23 also exhibits sensitivity to data characteristics and sampling strategy. In particular, its prediction accuracy degrades as the sampling ratio decreases from 50% to 10%. This effect is especially pronounced for the Hurricane dataset, where strong spatial variability in data values reduces the effectiveness of uniform sampling.

We also note that, although Tao19 and Jin22 partially support PSNR and SSIM estimation, the current implementation in LibPressio-Predict [64] does not provide support for these metrics. As a result, our comparison with these methods is limited to CR prediction. Regarding Mumenin et al. [35], while it is the closest prior work in terms of objective and prediction framework, a direct comparison is not entirely equivalent. Their method is designed for 2D data and focuses on the SZ2 compressor, whereas DeepCQ targets a broader setting, including 3D scientific data and multiple compressors. These differences in scope and application domain should be considered when interpreting the comparative results.

E. Model training overhead

■ ***Training overhead and amortization:*** We compare the training cost of DeepCQ with the conventional approach used in prior work [35]. In the conventional approach, a separate model is trained for each combination of data field, compressor, and quality metric. For the Nyx application, training one complete model, including the feature extraction backbone and prediction head, takes approximately 0.71 hours. In contrast, DeepCQ trains a shared backbone once, which takes 3.69 hours, and then trains lightweight metric prediction heads, each of which takes only 0.33 hours. The same backbone is reused across all configurations.

This two-stage design reduces the total training cost as the number of data fields, compressors, and quality metrics increases. For 4 fields, 5 compressors, and 3 quality metrics, the conventional end-to-end approach requires 42.6 hours of training, while DeepCQ requires 23.49 hours. This results in a 1.81× training speedup.

The speedup mainly comes from sharing the cost of backbone training across all prediction tasks. Instead of training the full model again for every combination of field, compressor, and metric, DeepCQ trains the backbone once and only trains a lightweight prediction head for each task. Therefore, the improvement comes from reusing the backbone, not from the size of the backbone itself.

TABLE V: MAPE of CR prediction with or without MoE.

Fields	SZ		SZ3		SZX		ZFP		SPERR	
	B	M	B	M	B	M	B	M	B	M
BD	8.17	5.33	9.31	5.82	11.20	9.64	11.57	9.45	13.11	11.05
DMD	4.68	4.96	8.05	3.54	5.98	3.10	2.61	5.07	1.66	3.96
VX	1.31	1.38	3.73	2.20	4.03	1.89	3.15	2.84	2.72	4.04
TEMP	5.45	3.54	3.18	3.13	3.23	2.34	5.54	4.51	4.41	1.51

B and M represent the Pred-NN with simple MLP and MoE, respectively.

The training cost is further recovered through the much lower inference time. In the same configuration space, GPU-based DeepCQ takes approximately 1.1 seconds to evaluate one configuration, while the compression-based baseline takes approximately 402 seconds. As a result, the full model training cost is recovered after about 210 configuration evaluations. In practical workflows with multiple fields, timesteps, and candidate configurations, this break-even point can be reached quickly.

The two-stage design also reduces model storage requirements. The shared backbone contains 117.36 million parameters and requires approximately 448.3 MB of storage. Each prediction head contains 8.73 million parameters and requires approximately 33.3 MB. The total model storage is therefore 448.3 + 33.3X MB, where X is the number of field, compressor, and metric combinations. For Nyx, with 4 fields, 5 compressors, and 3 metrics, X would be 60. This gives a total model size of approximately 2.45 GB, which is within the memory capacity of modern GPUs.

The primary application-level benefit is therefore not necessarily a reduction in end-to-end runtime, but the ability to perform compression-quality assessment within the latency constraints of online workflows. By separating configuration evaluation from actual compression execution, DeepCQ can evaluate many candidate configurations without repeatedly invoking the compressors.

F. Ablation Study

■ ***Effectiveness of MoE:*** Table V compares the MAPE of CR prediction with and without the MoE architecture. Overall, the MoE-based model achieves consistently lower prediction errors than the simple MLP model as the final prediction layer, demonstrating enhanced performance on data exhibiting varying compression quality. As discussed in Section III-E, integrating MoE into the Pred-NN enables the model to capture regime-dependent variations in the data more effectively.

While the MoE-based model exhibits slightly higher errors in a few edge cases (e.g., CR prediction for SPERR on the Velocity x field), it maintains strong overall performance, achieving lower MAPE in the majority of scenarios. These results confirm the effectiveness and robustness of the MoE architecture on scientific data manifesting regional variation in compression quality.

We further evaluate the MoE architecture using 2, 4, 6, and 8 expert heads, with compression-ratio prediction on the Baryon Density field as a representative case. The corresponding MAPE values are 5.56%, 4.65%, 4.40%, and 5.29%, respectively. Although the six-expert configuration achieves the lowest prediction error, its improvement over the four-expert configuration is relatively small, while the associated training and inference costs increase with the number of experts. This overhead becomes more substantial when the model is trained across multiple compressors and scientific fields. We therefore select four expert heads as a practical trade-off between prediction accuracy and computational efficiency.

When evaluated under a previously unseen physical regime, the MoE-based model exhibits gradual performance degradation rather than catastrophic failure, as shown in Fig. 6. Scientific fields belonging to the same application often share correlated physical structures, while Pred-NN learns a continuous mapping from data features and error bounds to compression quality. This allows the model to infer compression behavior for an unseen regime from nearby regions represented in the training data, thereby providing a degree of generalization across related physical regimes.

■ ***Impact of Model Capacity and Backbone Freezing:*** We compare DeepCQ with QualityNet [35], a lightweight two-dimensional compression-quality prediction model that we extend to three-dimensional scientific data, referred to as QualityNet-3D. For SZ3 compression-ratio prediction, QualityNet-3D achieves a MAPE of 35.03% on the Dark Matter Density field, compared with 3.54% for DeepCQ. Similarly, on the QRAIN field, QualityNet-3D obtains a MAPE of 26.87%, whereas DeepCQ achieves 8.15%. These results demonstrate that DeepCQ provides substantially more accurate compression-ratio predictions across both evaluated fields.

To further examine the effect of backbone capacity under the same two-stage training procedure, we compare ResNet-152-3D, which contains 117.36 million parameters, with a lightweight five-branch 3D convolutional encoder containing 27.6 million parameters and using kernel sizes of 3, 5, 7, 9, and 11. For SZ compression-ratio prediction on the Dark Matter Density field, the lightweight encoder achieves a MAPE of 12.46%, whereas ResNet-152-3D achieves 4.96%. This result indicates that the higher-capacity backbone learns more effective representations for compression-quality prediction.

We further compare the frozen-backbone strategy with full end-to-end training from scratch on the Dark Matter Density field using the same model architecture and experimental setup. The proposed strategy uses a pretrained ResNet-152 backbone with frozen parameters, whereas the end-to-end baseline trains the entire network from scratch. For SZ3 compression-ratio prediction, end-to-end training yields a MAPE of 10.11%, compared with 3.54% for the frozen-backbone strategy. This performance gap may be attributed to the large capacity of ResNet-152 relative to the limited amount of task-specific training data, which makes learning effective representations from scratch more challenging. Consequently, the frozen-backbone strategy reduces training cost while achieving higher prediction accuracy.

V. Limitations and Future Work

While the MoE-based design improves robustness to regime-dependent variation, several limitations remain. First, `DeepCQ` is sensitive to temporal distribution shift. Evolving simulation patterns can change the relationship among data features, error bounds, and quality metrics, resulting in increased prediction error. Second, `DeepCQ` is highly data-dependent. Changes in resolution or physical parameters that alter compression performance will impact the prediction accuracy and retraining will be needed. Third, cross-field generalization is limited because different fields can exhibit distinct compression performance, requiring separate prediction heads training for each data field.

These limitations reflect broader challenges in supervised learning under heterogeneous scientific data, including temporal shift, domain generalization, and field-level distribution mismatch that need to be investigated in future work.

VI. Conclusion

This paper presents `DeepCQ`, a two-stage deep-surrogate framework for efficient compression quality prediction. By decoupling application-specific feature extraction from compressor- and metric-specific prediction, `DeepCQ` reuses a shared backbone across lightweight downstream heads within the scientific application, achieving accurate and efficient compression quality prediction.

Experiments on Nyx, Hurricane, and Miranda show that `DeepCQ` accurately predicts CR, PSNR, and SSIM across five widely used error-bounded lossy compressors, achieving less than 10% average prediction error in 158 out of 165 cases. The modular design reduces training overhead by $1.81\times$ compared with monolithic per-task models, while the MoE-based prediction head improves robustness under regime-dependent compression behavior. Beyond accuracy, `DeepCQ` enables fast online compression decision-making. GPU-based inference achieves approximately 200 MB/s configuration-selection throughput and correctly identifies the best-fit compressor in 153 out of 160 configurations without performing compression operations during selection.

Overall, `DeepCQ` provides a practical foundation for scalable surrogate-based compression quality assessment in HPC workflows, with temporal distribution shift and cross-field generalization remaining key directions for future work.

VII. Acknowledgment

We sincerely thank the anonymous reviewers for their valuable feedback. This material was supported by the U.S. Department of Energy, Office of Science, Advanced Scientific Computing Research (ASCR), under Contract DE-AC02-06CH11357, and by the National Science Foundation under Grants OAC-2311875, OAC-2451636, OAC-2514034, OAC-2514036, OAC-2609480, CCF-2412345, and CCF-2521613. We also acknowledge the support provided by the startup fund from the University of North Carolina at Charlotte.

REFERENCES

[1] J. W. Hurrell, M. M. Holland, P. R. Gent, S. Ghan, J. E. Kay, P. J. Kushner, J. F. Lamarque, W. G. Large, D. Lawrence, K. Lindsay, W. H. Lipscomb, M. C. Long, N. Mahowald, D. R. Marsh, R. B. Neale, P. Rasch, S. Vavrus, M. Vertenstein, D. Bader, W. D. Collins, J. J. Hack, J. Kiehl, and S. Marshall, "The community earth system model: A framework for collaborative research," *Bulletin of the American Meteorological Society*, vol. 94, pp. 1339–1360, 9 2013. [Online]. Available: https://journals.ametsoc.org/view/journals/bams/94/9/bams-d-12-00121.1.xml

[2] D. Wang, J. Pulido, P. Grosset, S. Jin, J. Tian, K. Zhao, J. Ahrens, and D. Tao, "Tac+: Optimizing error-bounded lossy compression for 3d amr simulations," *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 3, pp. 421–438, 2023.

[3] S. Di and F. Cappello, "Fast error-bounded lossy hpc data compression with sz," in *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2016, pp. 730–739.

[4] P. Lindstrom, "Fixed-rate compressed floating-point arrays," *IEEE Transactions on Visualization and Computer Graphics*, vol. 20, pp. 2674–2683, 12 2014.

[5] M. Ainsworth, O. Tugluk, B. Whitney, and S. Klasky, "Multilevel techniques for compression and reduction of scientific data—the multivariate case," *SIAM Journal on Scientific Computing*, vol. 41, no. 2, pp. A1278–A1303, 2019.

[6] D. Tao, S. Di, Z. Chen, and F. Cappello, "Significantly improving lossy compression for scientific data sets based on multidimensional prediction and error-controlled quantization," in *2017 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2017, pp. 1129–1139.

[7] X. Liang, S. Di, D. Tao, S. Li, S. Li, H. Guo, Z. Chen, and F. Cappello, "Error-controlled lossy compression optimized for high compression ratios of scientific datasets," in *2018 IEEE International Conference on Big Data (Big Data)*. IEEE, 2018, pp. 438–447.

[8] K. Zhao, S. Di, X. Liang, S. Li, D. Tao, Z. Chen, and F. Cappello, "Significantly Improving Lossy Compression for HPC Datasets with Second-Order Prediction and Parameter Optimization," in *HPDC 2020 - Proceedings of the 29th International Symposium on High-Performance Parallel and Distributed Computing*. Association for Computing Machinery, Inc, Jun. 2020, pp. 89–100. [Online]. Available: https://dl.acm.org/doi/10.1145/3369583.3392688

[9] K. Zhao, S. Di, M. Dmitriev, T.-L. D. Tonellot, Z. Chen, and F. Cappello, "Optimizing Error-Bounded Lossy Compression for Scientific Data by Dynamic Spline Interpolation," in *2021 IEEE 37th International Conference on Data Engineering (ICDE)*, 2021, pp. 1643–1654.

[10] X. Liang, Q. Gong, J. Chen, B. Whitney, L. Wan, Q. Liu, D. Pugmire, R. Archibald, N. Podhorszki, and S. Klasky, "Error-controlled, progressive, and adaptable retrieval of scientific data with multilevel decomposition," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2021, pp. 1–13.

[11] R. Ballester-Ripoll, P. Lindstrom, and R. Pajarola, "Tthresh: Tensor compression for multidimensional visual data," *IEEE Transactions on Visualization and Computer Graphics*, vol. 26, no. 9, pp. 2891–2903, 2020.

[12] M. Burtscher and P. Ratanaworabhan, "Fpc: A high-speed compressor for double-precision floating-point data," *IEEE Transactions on Computers*, vol. 58, no. 1, pp. 18–31, 2009.

[13] Z. Wang, A. Bovik, H. Sheikh, and E. Simoncelli, "Image quality assessment: from error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.

[14] A. H. Baker, A. Pinard, and D. M. Hammerling, "Dssim: a structural similarity index for floating-point data," *arXiv preprint arXiv:2202.02616*, 2 2022. [Online]. Available: https://arxiv.org/abs/2202.02616v2

[15] R. Underwood, J. C. Calhoun, S. Di, A. Apon, and F. Cappello, "OptZConfig: Efficient Parallel Optimization of Lossy Compression Configuration," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 12, pp. 3505–3519, Dec. 2022. [Online]. Available: https://ieeexplore.ieee.org/document/9730115/

[16] X. Liang, K. Zhao, S. Di, S. Li, R. Underwood, A. M. Gok, J. Tian, J. Deng, J. C. Calhoun, D. Tao, Z. Chen, and F. Cappello, "Sz3: A modular framework for composing prediction-based error-bounded lossy compressors," *IEEE Transactions on Big Data*, vol. 9, no. 2, pp. 485–498, 2023.

[17] A. M. Gok, S. Di, Y. Alexeev, D. Tao, V. Mironov, X. Liang, and F. Cappello, "Pastri: Error-bounded lossy compression for two-electron integrals in quantum chemistry," in *2018 IEEE international conference on cluster computing (CLUSTER)*. IEEE, 2018, pp. 1–11.

[18] P. G. Lindstrom, "Fpzip," Lawrence Livermore National Laboratory (LLNL), Livermore, CA (United States), Tech. Rep., 2017.

[19] D. Tao, S. Di, X. Liang, Z. Chen, and F. Cappello, "Optimizing lossy compression rate-distortion from automatic online selection between sz and zfp," *IEEE Transactions on Parallel and Distributed Systems*, vol. 30, no. 8, pp. 1857–1871, 2019.

[20] S. Jin, S. Di, J. Tian, S. Byna, D. Tao, and F. Cappello, "Improving prediction-based lossy compression dramatically via ratio-quality modeling," in *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 2022, pp. 2494–2507.

[21] A. Khan, S. Di, K. Zhao, J. Liu, K. Chard, I. Foster, and F. Cappello, "SECRE: Surrogate-Based Error-Controlled Lossy Compression Ratio Estimation Framework," in *2023 IEEE 30th International Conference on High Performance Computing, Data, and Analytics (HiPC)*, 2023, pp. 132–142.

[22] K. Zhao, S. Di, M. Dmitriev, T. L. D. Tonellot, Z. Chen, and F. Cappello, "Optimizing error-bounded lossy compression for scientific data by dynamic spline interpolation," *Proceedings - International Conference on Data Engineering*, vol. 2021-April, pp. 1643–1654, 4 2021.

[23] J. Liu, S. Di, K. Zhao, X. Liang, S. Jin, Z. Jian, J. Huang, S. Wu, Z. Chen, and F. Cappello, "High-performance effective scientific error-bounded lossy compression with auto-tuned multi-component interpolation," *Proc. ACM Manag. Data*, vol. 2, no. 1, Mar. 2024. [Online]. Available: https://doi.org/10.1145/3639259

[24] X. Yu, S. Di, K. Zhao, J. Tian, D. Tao, X. Liang, and F. Cappello, "Szx: an ultra-fast error-bounded lossy compressor for scientific datasets," 2022.

[25] S. Li, P. Lindstrom, and J. Clyne, "Lossy scientific data compression with sperr," in *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2023, pp. 1007–1017.

[26] W. A. Pearlman, A. Islam, N. Nagaraj, and A. Said, "Efficient, low-complexity image coding with a set-partitioning embedded block coder," *IEEE transactions on circuits and systems for video technology*, vol. 14, no. 11, pp. 1219–1235, 2004.

[27] X. Tang and W. A. Pearlman, "Three-dimensional wavelet-based compression of hyperspectral images," in *Hyperspectral data compression*. Springer, 2006, pp. 273–308.

[28] T. Lu, Q. Liu, X. He, H. Luo, E. Suchyta, J. Choi, N. Podhorszki, S. Klasky, M. Wolf, T. Liu, and Z. Qiao, "Understanding and modeling lossy compression schemes on hpc scientific data," in *2018 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2018, pp. 348–357.

[29] J. Wang, T. Liu, Q. Liu, X. He, H. Luo, and W. He, "Compression ratio modeling and estimation across error bounds for lossy compression," *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 7, pp. 1621–1635, 2020.

[30] J. Wang, Q. Chen, T. Liu, Q. Liu, and X. He, "zperf: A statistical gray-box approach to performance modeling and extrapolation for scientific lossy compression," *IEEE Transactions on Computers*, vol. 72, no. 9, pp. 2641–2655, 2023.

[31] Z. Qin, J. Wang, Q. Liu, J. Chen, D. Pugmire, N. Podhorszki, and S. Klasky, "Estimating lossy compressibility of scientific data using deep neural networks," *IEEE Letters of the Computer Society*, vol. 3, no. 1, pp. 5–8, 2020.

[32] R. Underwood, J. Bessac, D. Krasowska, J. C. Calhoun, S. Di, and F. Cappello, "Black-box statistical prediction of lossy compression ratios for scientific data," *The International Journal of High Performance Computing Applications*, vol. 37, no. 3-4, pp. 412–433, 2023.

[33] D. Krasowska, J. Bessac, R. Underwood, J. C. Calhoun, S. Di, and F. Cappello, "Exploring lossy compressibility through statistical correlations of scientific datasets," in *2021 7th International Workshop on Data Analysis and Reduction for Big Scientific Data (DRBSD-7)*. IEEE, 2021, pp. 47–53.

[34] A. Ganguli, R. Underwood, J. Bessac, D. Krasowska, J. C. Calhoun, S. Di, and F. Cappello, "A lightweight, effective compressibility estimation method for error-bounded lossy compression," in *2023 IEEE International Conference on Cluster Computing (CLUSTER)*, 2023, pp. 247–258.

[35] K. M. Mumenin, D. Dai, J. Wang, and S. Di, "Qualitynet: Error-bounded lossy compression quality prediction via deep surrogate," in *2024 IEEE International Conference on Big Data (BigData)*. IEEE, 2024, pp. 252–261.

[36] R. Underwood, V. Malvoso, J. C. Calhoun, S. Di, and F. Cappello, "Productive and performant generic lossy data compression with libpressio," in *2021 7th International Workshop on Data Analysis and Reduction for Big Scientific Data (DRBSD-7)*. IEEE, 2021, pp. 1–10.

[37] M. Pedersen, J. Y. Hardeberg *et al.*, "Full-reference image quality metrics: Classification and evaluation," *Foundations and Trends® in Computer Graphics and Vision*, vol. 7, no. 1, pp. 1–80, 2012.

[38] S.-H. Gao, M.-M. Cheng, K. Zhao, X.-Y. Zhang, M.-H. Yang, and P. Torr, "Res2Net: A New Multi-Scale Backbone Architecture," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 2, pp. 652–662, 2021.

[39] S. Lu, Y. Ding, M. Liu, Z. Yin, L. Yin, and W. Zheng, "Multiscale feature extraction and fusion of image and text in vqa," *International Journal of Computational Intelligence Systems*, vol. 16, no. 1, p. 54, 2023.

[40] H. Guo, L. Shi, and J. Liu, "An improved multi-scale feature extraction network for medical image segmentation," *Quantitative Imaging in Medicine and Surgery*, vol. 14, no. 12, pp. 8 331 346–8 338 346, 2024.

[41] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *International Conference on Medical image computing and computer-assisted intervention*. Springer, 2015, pp. 234–241.

[42] S. Bhojanapalli, A. Chakrabarti, D. Glasner, D. Li, T. Unterthiner, and A. Veit, "Understanding robustness of transformers for image classification," *CoRR*, vol. abs/2103.14586, 2021. [Online]. Available: https://arxiv.org/abs/2103.14586

[43] A. Arnab, M. Dehghani, G. Heigold, C. Sun, M. Lučić, and C. Schmid, "Vivit: A video vision transformer," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 6836–6846.

[44] C. Forigua, M. Escobar, and P. Arbelaez, "Superformer: Volumetric transformer architectures for mri super-resolution," in *International workshop on simulation and synthesis in medical imaging*. Springer, 2022, pp. 132–141.

[45] S. Kornblith, J. Shlens, and Q. V. Le, "Do better ImageNet models transfer better?" in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 2661–2671.

[46] N. Tajbakhsh, J. Y. Shin, S. R. Gurudu, R. T. Hurst, C. B. Kendall, M. B. Gotway, and J. Liang, "Convolutional neural networks for medical image analysis: Full training or fine tuning?" *IEEE Transactions on Medical Imaging*, vol. 35, no. 5, pp. 1299–1312, 2016.

[47] Y. Gorishniy, I. Rubachev, and A. Babenko, "On embeddings for numerical features in tabular deep learning," in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 24 991–25 004. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/9e9f0ffc3d836836ca96cbf8fe14b105-Paper-Conference.pdf

[48] S. R. Stahlschmidt, B. Ulfenborg, and J. Synnergren, "Multimodal deep learning for biomedical data fusion: a review," *Briefings in Bioinformatics*, vol. 23, no. 2, p. bbab569, 01 2022. [Online]. Available: https://doi.org/10.1093/bib/bbab569

[49] J. Bromley, I. Guyon, Y. LeCun, E. Säckinger, and R. Shah, "Signature verification using a "siamese" time delay neural network," in *Advances in Neural Information Processing Systems*, J. Cowan, G. Tesauro, and J. Alspector, Eds., vol. 6. Morgan-Kaufmann, 1993. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/1993/file/288cc0ff022877bd3df94bc9360b9c5d-Paper.pdf

[50] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. V. Le, G. E. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," *CoRR*, vol. abs/1701.06538, 2017. [Online]. Available: http://arxiv.org/abs/1701.06538

[51] J. Puigcerver, C. Riquelme, B. Mustafa, and N. Houlsby, "From sparse to soft mixtures of experts," 2024. [Online]. Available: https://arxiv.org/abs/2308.00951

[52] W. Cai, J. Jiang, F. Wang, J. Tang, S. Kim, and J. Huang, "A survey on mixture of experts in large language models," *IEEE Transactions on Knowledge and Data Engineering*, p. 1–20, 2025. [Online]. Available: http://dx.doi.org/10.1109/TKDE.2025.3554028

[53] "Quick Compression Analysis Toolkit (QCAT)," Online. Available: https://github.com/szcompressor/qcat, 2023, accessed: Mar. 10, 2025.

[54] "Scientific Data Reduction Benchmarks," Online. Available: https://sdrbench.github.io, 2024, accessed: Mar. 10, 2025.

[55] A. S. Almgren, J. B. Bell, M. J. Lijewski, Z. Lukić, and E. Van Andel, "Nyx: A massively parallel amr code for computational cosmology," *The Astrophysical Journal*, vol. 765, no. 1, p. 39, feb 2013. [Online]. Available: https://dx.doi.org/10.1088/0004-637X/765/1/39

[56] N. G. S. F. Center, "Hurricane isabel simulation data," NASA Cloud-Hosted Data, 2004, accessed: 2024-08-26. [Online]. Available: https://www.nccs.nasa.gov/services/data-collections-and-simulation/Hurricane-Isabel

[57] W. H. Cabot and A. W. Cook, "Reynolds number effects on rayleigh–taylor instability with possible implications for type ia supernovae," *Nature Physics*, vol. 2, no. 8, pp. 562–568, 2006.

[58] X. Zhu, C. Vondrick, C. C. Fowlkes, and D. Ramanan, "Do we need more training data?" *CoRR*, vol. abs/1503.01508, 2015. [Online]. Available: http://arxiv.org/abs/1503.01508

[59] I. Kandel and M. Castelli, "The effect of batch size on the generalizability of the convolutional neural networks on a histopathology dataset," *ICT Express*, vol. 6, no. 4, pp. 312–315, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2405959519303455

[60] B. Friesen, A. Almgren, Z. Lukić, G. Weber, D. Morozov, V. Beckner, and M. Day, "In situ and in-transit analysis of cosmological simulations," *Computational Astrophysics and Cosmology 2016 3:1*, vol. 3, no. 1, Aug. 2016.

[61] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *Science*, vol. 313, no. 5786, pp. 504–507, 2006.

[62] Y. Bengio, A. Courville, and P. Vincent, "Representation learning: A review and new perspectives," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 8, pp. 1798–1828, 2013.

[63] C. Baur, B. Wiestler, S. Albarqouni, and N. Navab, "3d medical image segmentation with labeled and unlabeled data using autoencoders at the example of liver segmentation in ct images," *arXiv preprint arXiv:2003.07923*, 2020.

[64] R. Underwood, V. Malvoso, J. C. Calhoun, S. Di, and F. Cappello, "Productive and performant generic lossy data compression with libpressio," in *2021 7th International Workshop on Data Analysis and Reduction for Big Scientific Data (DRBSD-7)*. IEEE, 2021, pp. 1–10.